



Tools & Agents

in GitHub Copilot

From Built-in Tools to Autonomous Agent Workflows

Deccansoft Software Services

GH-300 Certified GitHub Copilot Expert Training

Agenda



1. What is an Agent?



2. Tools: Built-in, MCP & Extension Tools



3. Using Tools in Prompts



4. Local Agent Modes: Agent, Plan, Ask, Custom



5. Session Targets: Local, Copilot CLI & Cloud



6. Permissions: Default, Bypass & Autopilot



7. Workspace vs Worktree Isolation



8. Local Agent vs Copilot CLI Differences



9. Agent Handoff Workflow

What is an Agent?



An AI assistant that works autonomously to complete tasks

Example: "Add login functionality to my app"

Autonomous Execution

Takes a high-level goal and breaks it into steps without constant guidance

Multi-File Editing

Identifies and edits files across the entire project automatically

Tool Invocation

Uses built-in, MCP, and extension tools to complete tasks

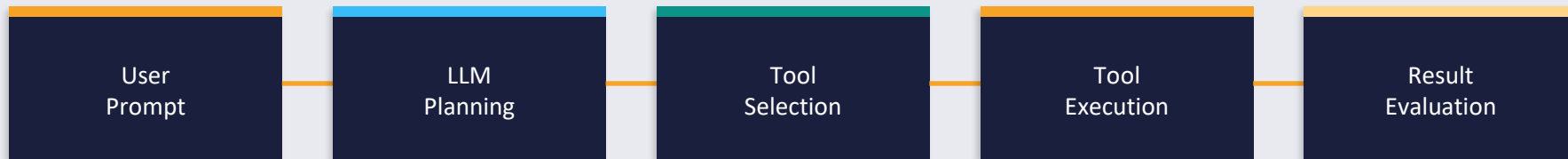
Self-Correction

Monitors results, detects errors, and iterates to fix them

Decision Making

Chooses appropriate tools and approaches based on context

The Agent Loop – How Agents Work



Self-correction loop: If errors detected, re-plan and re-execute

What the Agent Loop Sends to the LLM:

- System prompt with agent instructions
- Conversation history & user prompt
- Tool definitions (built-in + MCP + extensions)
- Machine context (OS, workspace info)
- Custom instructions (.github/copilot-instructions.md)
- Previous tool results & error outputs

Tools – Three Types of Agent Tools

Tools extend agents with specialized functionality for accomplishing tasks



Built-in Tools

Pre-installed in VS Code.
Always available.

- `read_file`, `edit_file`
- `run_in_terminal`
- `search/codebase`
- `read/problems`
- `web/fetch`



MCP Tools

External via Model
Context Protocol.

- GitHub MCP Server
- Figma MCP Server
- Database tools
- Custom APIs
- Third-party services



Extension Tools

From VS Code
extensions.

- Test runners
- Debuggers
- Linters
- Language servers
- Custom extensions

Built-in Tools — Core Capabilities

Editing	#editFiles	Create, modify, or delete files in workspace
Terminal	#runCommands	Run shell commands, install packages, build, test
Search	#codebase	Semantic search across project files
Search	#changes	Search recent code changes and diffs
Search	#usages	Find all references/usages of a symbol
Diagnostics	#problems	Get compiler/linter errors from the editor
Web	#fetch	Fetch content from URLs for context
Navigation	#findSymbol	Language-aware symbol navigation (C#, TS, etc.)

MCP Tools — Model Context Protocol

MCP is an open standard enabling AI models to interact with external tools and services

Key Benefits

- Extended context from external data sources
- Reduced manual effort via automation
- Seamless integration across platforms
- No custom integrations needed
- Unified interface for all tools; BYOR registry allowlist (Apr 2026)

Configuration (mcp.json)

```
{
  "servers": {
    "github": {
      "command": "npx",
      "args": [
        "-y", "@modelcontextprotocol",
        "/server-github"
      ],
      "env": {
        "GITHUB_TOKEN": "${env:TOKEN}"
      }
    }
  }
}
```

Popular MCP Servers:

[GitHub](#)[Figma](#)[Slack](#)[Jira](#)[Database](#)[Docker](#)

Extension Tools – VS Code Extensions

Extensions can contribute tools that the agent can invoke during task execution

Test Runners

Run and analyze unit/integration tests

Example: `@test` agent in Visual Studio

Debuggers

Debug sessions, breakpoints, profiling

Example: `@debug`, `@profiler` agents

Language Servers

Symbol nav, type info, refactoring

Example: `find_symbol` for C#, TypeScript

IDE Integration

VS-specific features and commands

Example: `@vs` agent in Visual Studio

How Tools Are Used in Prompts

Enabling Tools

1. Use the Tools Picker (⚙️) in Chat view
2. Type # in chat to see available tools
3. Enable/disable per-request or per-session
4. MCP tools: disabled by default, opt-in
5. Max 128 tools enabled per request

Using Tools in Prompts

```
#fetch "https://api.example.com/data"

#codebase find auth middleware

#changes show recent modifications

#problems list current errors

Agent decides tools automatically
```

Tool Sets — Group Related Tools

```
// Custom tool set in settings
{
  "reader": {
    "tools": ["search/changes", "search/codebase", "read/problems", "search/usages"],
    "description": "Tools for reading and gathering context",
    "icon": "book"
  }
}

// Use in prompt: #reader analyze auth flow
```

Local Agent Modes

Four interaction modes for the local agent in VS Code Chat (Agent mode is also GA on JetBrains, Mar 2026)



Agent

Autonomous multi-step execution. Edits files, runs commands, iterates on errors until task is complete.

Best for: Complex tasks, full implementation



Plan

Creates a detailed plan/blueprint before coding. Review and approve the plan first, then implement.

Best for: Complex features needing design review



Ask

Conversational Q&A mode. Answers questions, explains code, but does NOT modify files.

Best for: Learning, code exploration, quick answers



Custom

User-defined agents with specific roles, tools, and instructions via .agent.md files.

Best for: Specialized workflows (review, docs, test)

Agent Mode – Autonomous Pair Programmer



Agent mode iterates autonomously until the task is complete

Capabilities

- ✓ Analyzes codebase & identifies relevant files
- ✓ Creates and modifies files across project
- ✓ Runs terminal commands (build, test, install)
- ✓ Detects & auto-fixes compile/lint errors
- ✓ Monitors test output and iterates
- ✓ Proposes terminal commands for approval

Example Prompt

"Add user authentication with JWT tokens. Include login/register endpoints, password hashing, and middleware."

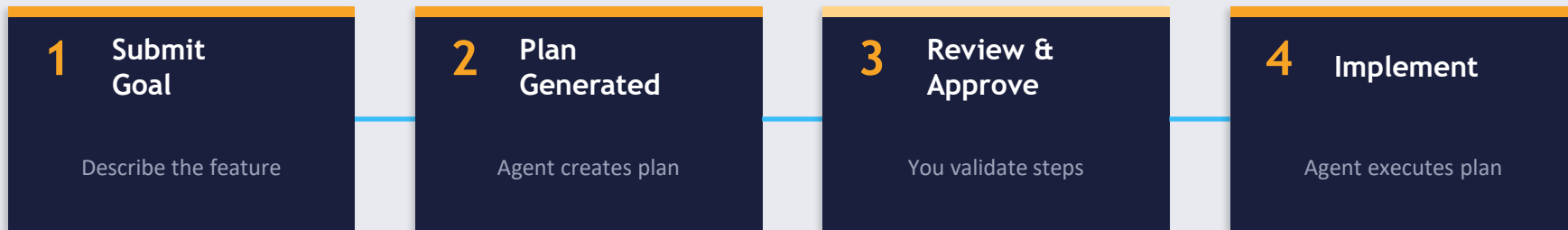
Agent will:

1. Search codebase for existing auth
2. Create auth controller + routes
3. Install bcrypt, jsonwebtoken
4. Create middleware files
5. Run tests & fix any failures

Plan Mode – Blueprint Before Build



Review and approve the blueprint before the agent starts coding



Plan Mode Handoff Options

Start Implementation

Execute plan in current local agent session

Continue in Copilot CLI

Hand off to background CLI for autonomous execution

Accept plan and build on Autopilot

Run the plan with full autonomy, no approvals needed

Ask Mode & Custom Agents

Ask Mode

Conversational Q&A — no file modifications

- Explain code concepts
- Answer questions about codebase
- Discuss architecture decisions
- Learn about APIs/frameworks
- No write access to files

Custom Agents

Define via .agent.md files in your repo

```
# .github/agents/reviewer.agent.md
---
name: Code Reviewer
description: Reviews code for quality
tools:
  - search/codebase
  - read/problems
---
```

You are a code reviewer. Focus on code quality, security, and style.

AGENTS.md — Project-wide Agent Instructions

Share project-specific instructions across all agents (local, CLI, cloud). Works like copilot-instructions.md but specifically for agent behavior, tool access, and coding conventions. Custom agents (.agent.md) also run on Copilot cloud agent and in Visual Studio 2026 (v18.4+), not just VS Code. Keeps all agents in sync with your team's practices.

Session Targets – Where Agents Run

Choose where and how your agent executes based on the task requirements

	Where	How	Tool Access	Parallel	Best For
 Local	VS Code editor	Interactive, real-time	All tools: built-in, MCP, extensions	No	Interactive coding, debugging, tasks needing extension tools <i>e.g. Refactor a function, fix a failing test, explore a codebase</i>
 Copilot CLI	Background on your machine	Autonomous, non-blocking	Limited built-in, local MCP only	Yes (multiple sessions)	Parallel tasks, well-scoped implementation, PoC variants <i>e.g. Run 3 implementations side-by-side, batch file migrations</i>
 Copilot Cloud Agent	GitHub's infrastructure	Fully autonomous, async	Cloud-only tools, GitHub Actions	Yes	Issue-to-PR, research, branch-only work, autonomous tasks, team collaboration <i>e.g. Auto-fix a GitHub issue, run research sessions, self-review + security scans, trigger from GitHub Mobile</i>

Permissions – Controlling Agent Autonomy

The permissions picker lets you choose the autonomy level for each session



Default Approvals

Manual confirmation for each tool call. Agent stops and asks before every file edit, terminal command, and external tool call.

Auto-approve tools	No
Auto-respond questions	No
Auto-iterate to complete	No



Bypass Approvals

All tool calls auto-approved, but agent still stops at decision points. Blocking questions still require human input.

Auto-approve tools	Yes
Auto-respond questions	No
Auto-iterate to complete	No



Autopilot (Preview)

Full autonomous execution. Auto-approves tools, auto-responds to questions, and iterates until task completion.

Auto-approve tools	Yes
Auto-respond questions	Yes
Auto-iterate to complete	Yes

Permissions – Commands & Configuration

VS Code Chat Commands

<code>/yolo</code> or <code>/autoApprove</code>	Enable auto-approval
<code>/disableYolo</code>	Disable auto-approval
Permissions Picker	Bottom of chat input area
<code>chat.tools.global.autoApprove</code> / <code>chat.tools.terminal.autoApprove</code>	Global auto-approve + per-command allow/deny
<code>chat.autopilot.enabled</code>	Enable Autopilot (on by default)

Copilot CLI Flags

<code>--autopilot</code>	Enable autopilot mode
<code>--allow-all</code>	Grant all permissions
<code>--max-autopilot-continues 10</code>	Cap autonomous steps
<code>--no-ask-user</code>	Suppress clarifying questions
<code>-p "prompt text"</code>	Pass prompt non-interactively

Organization Admin Policies

<code>ChatToolsAutoApprove</code>	Hide Bypass/Autopilot from developers
<code>ChatAgentMode</code>	Disable agent mode entirely
<code>ChatAgentExtensionTools</code>	Block extension-contributed tools
<code>ChatMCP</code>	Control MCP sources: BYOR registry, registryOnly, or off (Apr 2026)

Workspace vs Worktree Isolation

Two isolation modes for Copilot CLI sessions — choose based on risk tolerance

Aspect	Workspace Isolation	Worktree Isolation
Where changes apply	Directly in your current workspace	In a separate Git worktree folder
File conflicts	Risk of conflicts with your edits	No conflicts — fully isolated copy
Parallel sessions	Risky — shared working directory	Safe — each session has own worktree
Permissions available	Default, Bypass, Autopilot	Bypass only (auto-set, unchangeable)
Auto-commit	No automatic commits	Auto-commits at end of each turn
Review workflow	Changes visible immediately	Review & merge via Git branch/PR
Git requirement	No	Yes — must be a Git repository
Best for	Quick direct edits, simple tasks	Parallel agents, safe experimentation

Isolation Modes – Visual Comparison

Workspace Isolation

 Your Working Directory

You

Agent

⚠ Both edit same files — conflict risk!

Changes applied immediately
All 3 permission levels available

Worktree Isolation

 Your Dir

 Worktree

You

Agent

✓ Completely separate — no conflicts!

Auto-commits each turn
Bypass Approvals only (auto-set)

Local Agent vs Copilot CLI – Key Differences

Feature	Local Agent	Copilot CLI
Runs in	VS Code editor process	Background process (separate)
Blocks your editor?	Yes — foreground interaction	No — runs independently
Parallel sessions	One active session	Multiple simultaneous sessions
Built-in tool access	Full access to all tools	Limited (cannot access all VS Code tools)
Extension tools	Full access	Not available
MCP servers	All configured servers	Local MCP only (no auth required)
Model access	All available models	Limited to CLI-available models
Isolation modes	N/A (uses workspace directly)	Workspace or Worktree isolation
Survives VS Code restart?	No — tied to editor session	Yes — independent process
Best for	Interactive work, debugging	Autonomous background tasks

Choosing the Right Session Target

Decision guide: Local Agent vs Copilot CLI vs Cloud

Need extension tools or debugger?

Yes →

Local Agent

Need to keep working while agent runs?

Yes →

Copilot CLI

Need multiple parallel agents?

Yes →

**Copilot CLI
(Worktree)**

Task results in a Pull Request?

Yes →

Cloud Agent

Sensitive code that shouldn't leave machine?

Yes →

Local or CLI

Agent Handoff Workflow – Example

Hand off tasks between agent types to leverage their unique strengths

1



Plan with Local Agent

Use Plan mode: "Add OAuth2 login with Google provider"

Agent creates detailed implementation plan

2



Hand Off to Copilot CLI

Select "Continue in Copilot CLI" from Plan mode

Full conversation history carries over

3



CLI Executes Autonomously

Agent works in worktree isolation, auto-commits each turn

You continue working on other tasks in VS Code

4



Delegate to Copilot cloud agent

Use /delegate (or & shorthand) to hand off to Copilot cloud agent

Cloud agent self-reviews, runs security scans, and creates draft PR

Agent Handoff – In Practice

Step 1: Plan Mode (Local Agent)

User Prompt:

"Add unit tests for the auth module covering login, register, and password reset flows."

Copilot generates plan with:

- Files to create/modify, - Test cases per flow
- Dependencies needed

Step 2: Hand Off to Copilot CLI

Click "Start Implementation" dropdown

→ Select "Continue in Copilot CLI"

- Worktree (recommended for safety)
- Workspace (for quick changes)

Conversation history carries over!

Agent begins working autonomously.

Step 3: Delegate to Copilot cloud agent (/delegate)

In Copilot CLI session, once implementation is ready:

/delegate Create a PR with these test changes. Run the test suite in CI and fix any failures before requesting review.

Cloud Agent will:

1. Create a branch (copilot/add-auth-tests)
2. Push the changes from the worktree
3. Run CI/CD pipeline (GitHub Actions)
4. Fix any test failures autonomously
5. Open a draft Pull Request for team review

Common Confusion: Local Agent vs CLI + Isolation

? "Isn't Copilot CLI with Workspace Isolation the same as a Local Agent?"

No! Despite both editing your workspace files, they differ fundamentally:

- Local Agent runs inside VS Code's process — CLI runs as a separate background process
- Local Agent has full tool access (extensions, all MCP) — CLI has limited tools
- Local Agent blocks your editor while working — CLI lets you keep working
- Local Agent cannot run in parallel — CLI supports multiple simultaneous sessions

? "When should I use Local Agent vs Copilot CLI?"

Use Local Agent when:

You need debugger, test runners, or extension tools during the task

Use Copilot CLI when:

Task is well-scoped, you want to keep working, or need parallel execution

💡 Rule of Thumb

Start with Plan mode (Local Agent) to design the approach, then hand off to Copilot CLI for autonomous execution. If the result needs a PR, delegate to Cloud. This "Plan → CLI → Cloud" workflow combines the strengths of all three.

Key Takeaways



Agents are autonomous AI assistants that plan, execute, and self-correct coding tasks



Three tool types: Built-in (core), MCP (external services), Extensions (IDE features)



Four local modes: Agent (do), Plan (design first), Ask (Q&A), Custom (specialized)



Three session targets: Local (interactive), CLI (background), Copilot cloud agent (research + PR + self-review + security scans)



Three permission levels: Default (manual), Bypass (auto-approve), Autopilot (full auto)



Worktree isolation enables safe parallel agents; Workspace isolation for direct edits



Agent handoff: Plan locally → Execute in CLI → Delegate to Cloud for PRs

Thank You!

Tools & Agents in GitHub Copilot

Deccansoft Software Services

GH-300 Certified GitHub Copilot Expert Training